SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA43

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. **(L2)**

Assessment Tool: Data Interpretation (Medium)

Weightage: 20%

QUESTIONS: **Total Marks: 20**

1: HTTP Response Analysis

A web server returns the following HTTP response header:

```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive
```

Questions:

1. Identify any incorrect or invalid fields in the header.
2. Analyze the issue with `Content-Length` and explain its impact.
3. Determine whether the status code matches the response correctness.
4. Suggest corrections for the identified errors.

2: **DOM Structure Analysis**

Consider the following DOM tree representation:

```
<html>
  <head>
  <title>TestPage</title>
</head>
  <body>
  <div>
  <p>HelloWorld
  </div>
 </body>
 </html>
```

Questions:

1. Identify missing or improperly closed elements.
2. Analyze how the browser will interpret this structure.
3. Determine the impact on rendering and layout.
4. Suggest corrections to make the DOM valid.

3: GET vs POST in Login System

A login system uses two different methods:

Method	URL Example	Data Visibility	Security Level
GET	/login?user=admin&pass=1234	Visible in URL	Low
POST	/login	Hidden in body	Moderate

Questions:

1. Interpret the differences in data transmission between GET and POST.
2. Analyze which method is more suitable for login and why.
3. Identify security risks in using GET for authentication.
4. Suggest best practices for secure login implementation.

4: HTML Code Inspection

Analyze the following HTML snippet:

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome
<p>This is a paragraph
<img src="image.jpg">
</body>
</html>
```

Questions:

1. Identify missing closing tags.
2. Analyze issues with the `<img>` tag.
3. Determine how browsers will handle these errors.
4. Suggest a corrected version of the code.

5: Browser-Server Communication Flow

The following sequence describes a web request:

1. User enters a URL in the browser
2. DNS lookup occurs
3. HTTP request is sent
4. Server processes request
5. Response is returned
6. Browser renders page

Questions:

1. Interpret each step in the communication flow.
2. Identify where delays can occur.
3. Analyze the role of DNS in the process.
4. Suggest optimizations to improve performance.

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weigh tage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions
Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

CSA43 – INTERNET PROGRAMMING

ASSESSMENT TOOL 2: DATA INTERPRETATION (MEDIUM)

Detailed Solutions and Technical Analysis

Course Outcome CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery.

The solutions follow the five case studies and their sub-questions. Each answer identifies the issue, explains its effect, and recommends a correction or practical improvement.

1. HTTP Response Analysis

Given response header:

```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive
```

1.1 Identify incorrect or invalid fields

The clearly invalid field is Content-Length: -150. Content-Length represents the size of the message body in octets (bytes) and cannot be negative. The other listed fields are syntactically plausible: HTTP/1.1 200 OK is a success status line, Content-Type: text/html identifies an HTML response, and Connection: keep-alive indicates that the connection may be reused. The supplied Date value should also be checked: the weekday/date combination appears inconsistent, so it should be replaced with the actual current HTTP date.

1.2 Analyze Content-Length and its impact

A negative Content-Length is invalid because a message body cannot have a negative number of bytes. It can cause the response to be rejected or handled inconsistently by clients, intermediaries, or validation tools. Incorrect message framing can interfere with determining where a response body ends, particularly when persistent connections are reused. The server must send an accurate non-negative length when Content-Length is used.

1.3 Does status code 200 match response correctness?

200 OK indicates that the request succeeded; it does not guarantee that every header is valid or that the response is correctly framed. Therefore, the status code can describe the intended outcome, but the complete response shown is malformed because of the invalid Content-Length. A client may still report a protocol or parsing error despite the 200 status line.

1.4 Suggested corrections

Replace -150 with the actual response-body size in bytes. Do not guess a number; calculate it from the exact bytes sent. For example, if the HTML body is exactly 150 bytes, the corrected field is Content-Length: 150. If the server does not know the length in advance, it should use valid HTTP/1.1 message framing, such as chunked transfer coding, rather than sending a negative length.

```
HTTP/1.1 200 OK
Date: <current valid HTTP-date>
Content-Type: text/html
Content-Length: 150
Connection: keep-alive
```

The value 150 is illustrative and is correct only if the actual body is 150 bytes. Replace the Date placeholder with the actual date in valid HTTP-date format.

2. DOM Structure Analysis

Provided structure:

```
<html>
  <head>
    <title>TestPage</title>
  </head>
  <body>
    <div>
      <p>HelloWorld
```

```
    </div>
  </body>
</html>
```

2.1 Identify missing or improperly closed elements

The <p> element is opened but not explicitly closed before the closing tag. The , <body>, <head>, and <html> elements have closing tags. In HTML, browsers may implicitly close a paragraph when a block-level element such as begins, but the supplied markup should still be corrected explicitly.

2.2 How will the browser interpret it?

An HTML browser uses its parsing rules to construct a DOM tree. When it encounters the closing while a paragraph is still open, the parser generally closes the paragraph implicitly before closing the div. The browser is therefore likely to display the text rather than stop rendering. Error recovery does not make the source well-structured, and the resulting DOM may differ from what the author intended.

2.3 Impact on rendering and layout

For this simple example, the visible impact may be small because browsers automatically repair many HTML syntax errors. In larger documents, invalid nesting may affect CSS selectors, layout, scripts that traverse the DOM, and accessibility. Explicitly closing elements makes the structure predictable and easier to maintain.

2.4 Corrected structure

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <title>TestPage</title>
  </head>
  <body>
    <div>
      <p>HelloWorld</p>
    </div>
  </body>
</html>
```

This version explicitly closes the paragraph and includes a document type, language declaration, and character encoding.

3. GET vs POST in Login System

The assessment compares GET /login?user=admin&pass=1234, where credentials are visible in the URL, with POST /login, where form data is sent in the request body.

3.1 Differences in data transmission

GET sends parameters in the URL query string in the given example. URLs can be stored in browser history, server logs, proxy logs, analytics systems, and may be included in referrer information depending on the situation. POST sends form fields in the request body, so values are not ordinarily shown in the URL. However, request bodies can still be logged or exposed if systems are misconfigured.

3.2 Which method is more suitable for login?

POST is more suitable for a conventional login form because it avoids placing usernames and passwords in the URL. POST itself does not encrypt data; the assessment's "Moderate" security label is only a relative comparison. A secure login must use HTTPS, regardless of whether GET or POST is selected.

3.3 Security risks of GET for authentication

Using GET for credentials can expose them through browser history, URL logging, screenshots, shared links, monitoring tools, and referrer leakage. URLs may be copied or bookmarked, creating further opportunities for accidental disclosure. Credentials in a URL should therefore be avoided.

3.4 Best practices for secure login

- Submit credentials using POST over HTTPS; redirect HTTP traffic to HTTPS and configure secure TLS.
- Never place passwords, access tokens, or other secrets in URLs.
- Validate credentials on the server and store passwords using a strong, salted password-hashing algorithm rather than plaintext.

- Use rate limiting and suitable protections against brute-force attempts; avoid revealing whether only the username or password was incorrect.
- Use secure session cookies with HttpOnly, Secure, and an appropriate SameSite setting; rotate session identifiers after login.
- Apply multi-factor authentication where appropriate and protect login flows against CSRF where relevant.

Conclusion: POST is the better choice for login form submission, but HTTPS and server-side authentication safeguards are essential for actual security.

4. HTML Code Inspection

Provided snippet:

```
<!DOCTYPE html>
<html>
<head>
  <title>Sample</title>
</head>
<body>
  <h1>Welcome
  <p>This is a paragraph
  <img src="image.jpg">
</body>
</html>
```

4.1 Identify missing closing tags

The <h1> and <p> elements are not explicitly closed. The <img> element is a void element in HTML, so it does not require a closing tag. The </body> and </html> tags are present.

4.2 Analyze issues with the <img> tag

The image element has a src attribute but no alt attribute. The alt attribute provides a text alternative for users who cannot see the image and when the image cannot load. If the image conveys meaningful information, describe it concisely; if it is purely decorative, use alt="". The file image.jpg must also exist at the referenced location.

4.3 How will browsers handle these errors?

Browsers generally recover from omitted closing tags in HTML and may infer where headings and paragraphs end. The page may still render, but relying on error recovery can lead to an unexpected DOM structure and reduces maintainability. Missing alt text can also make the page less accessible.

4.4 Corrected HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Sample</title>
</head>
<body>
  <h1>Welcome</h1>
  <p>This is a paragraph.</p>
  <img src="image.jpg" alt="Description of the image">
</body>
</html>
```

Replace “Description of the image” with a meaningful description of the actual image. Use alt="" if the image is decorative and adds no information. The <img> element remains unclosed because HTML defines it as a void element.

5. Browser–Server Communication Flow

The assessment lists six stages: entering a URL, DNS lookup, sending an HTTP request, server processing, returning a response, and rendering the page.

5.1 Interpret each step

1. User enters a URL: The browser parses the URL to determine the scheme, hostname, port (if specified), and requested path.

2. DNS lookup: The browser or operating system resolves the hostname to an IP address, often checking caches before querying DNS resolvers.

3. HTTP request is sent: The browser establishes the required network connection (for HTTPS, a TLS handshake normally occurs) and sends a request containing the method, path, headers, and any applicable body.

4. Server processes request: The server routes the request, may run application logic or query a database, and prepares a response.

5. Response is returned: The server sends a status code, headers, and response body where applicable. The browser receives and interprets the response.

6. Browser renders page: The browser parses HTML into the DOM, processes CSS, runs applicable JavaScript, and requests dependent resources such as images, stylesheets, and fonts.

5.2 Where can delays occur?

Delays may occur during DNS resolution, establishing TCP or TLS connections, network transmission, server-side processing, database queries, large response downloads, and browser parsing or JavaScript execution. Images, fonts, scripts, redirects, and third-party resources can also delay the page becoming usable.

5.3 Role of DNS

DNS (Domain Name System) translates a human-readable hostname, such as www.example.com, into an IP address used to route network traffic. It allows users to access services by name rather than memorizing IP addresses. Cached DNS answers can reduce repeated lookup time. DNS resolution identifies an address; it does not itself deliver the web page.

5.4 Performance optimizations

- Use DNS caching and reliable DNS infrastructure to reduce repeated resolution delays.
- Use HTTPS with modern protocol support and connection reuse; avoid unnecessary redirects.
- Enable suitable HTTP caching headers and use a content delivery network (CDN) for static assets when appropriate.
- Compress text responses using Brotli or gzip and optimize images, fonts, and other large resources.
- Reduce unnecessary JavaScript and CSS, defer non-critical scripts, and remove unused resources.
- Optimize server code and database queries, monitor response times, and scale capacity when required.

Conclusion: Page-load performance depends on the entire request-response pipeline. Improvements should target the slowest stage, measured using browser developer tools and server-side monitoring.

Consolidated Findings

Case study	Main issue	Recommended solution
1. HTTP response	Negative Content-Length	Send accurate non-negative body length and valid headers.
2. DOM structure	Paragraph is not explicitly closed	Close before and use a clear structure.
3. Login methods	GET exposes credentials in URL	Use POST over HTTPS and secure server-side authentication.
4. HTML inspection	Missing </h1>, , and image alternative text	Close normal elements and provide appropriate alt text.
5. Communication flow	Possible bottlenecks at multiple stages	Measure and optimize DNS, connections, server, assets, and rendering.

Overall Conclusion

The five case studies show how HTTP response framing, DOM structure, secure form submission, valid HTML, and browser–server communication affect reliability, security, accessibility, and performance. A complete answer identifies each problem, explains its impact, and provides a standards-based correction. The original assessment rubric on page 4 emphasizes understanding of data, analysis, application of concepts, conclusions, and clarity; the solutions are organized to address those criteria.

Submission Checklist

- Fill in the student name and registration number on the original assessment pages.
- Review the reasoning and code examples for all five case studies.

- Replace the example image alt text with a description that matches the actual image, if using the corrected HTML.
- Complete the original academic integrity declaration and signature as required.